

Definición de Elementos

<!ELEMENT nombre EspContenido>

Ejemplos :

- <!ELEMENT hr EMPTY>
- <!ELEMENT documento ANY>
- <!ELEMENT region (provincia, extension, rio)>
- <!ELEMENT Nombre (#PCDATA)>
- <!ELEMENT chapter (introduccion, (p | quote | note)*, div*)>

EspContenido:

- EMPTY, indica que el elemento está vacío, sin contenido:
<!ELEMENT hr EMPTY>
- ANY, cualquier tipo de contenido. Es la especificación menos restrictiva, permite elementos hijos con o sin datos de caracteres entremezclados:
<!ELEMENT documento ANY>
- Listas de elementos hijos, nombres de elementos entre paréntesis (secuencias):
<!ELEMENT region (provincia, extension, rio)>
- #PCDATA (Parsed Character Data) contiene datos de caracteres y no se permiten elementos hijos:
<!ELEMENT nombre (#PCDATA)>
- Contenido mixto, con esta especificación de contenido, el elemento puede incluir cualquier cantidad de datos de caracteres, opcionalmente entremezclados con elementos hijos de los tipos especificados.
<!ELEMENT nombre (#PCDATA|hijo)*>

Cuando existen múltiples hijos tenemos los siguientes especificadores de contenido:

- **a+** - Una o más ocurrencias de a
<!ELEMENT libro (capitulo)+>
- **a*** - Cero o más ocurrencias de a
<!ELEMENT lista (objeto)*>
- **a?** - a o nada
<!ELEMENT mesa (plato)?>
- **a, b** - a seguido de b
<!ELEMENT resta (op1, op2)>
- **a | b** - a o b pero no ambos
<!ELEMENT precio (euros | pesetas)>
- **(expression)** - expresiones que pueden ser tomadas como una sola
<!ELEMENT capitulo (introduccion, (parrafo| comentario | nota)*, seccion*)>

Secuencias, es obligada la aparición de cada uno de los elementos enumerados y en el orden especificado:

```
<!ELEMENT nombre (nombre_propio, primer_apellido)>
<!ELEMENT nombre_propio (#PCDATA)>
<!ELEMENT primer_apellido (#PCDATA)>
.....
<nombre>
  <nombre_propio> Paul </nombre_propio>
  <primer_apellido> Newman </primer_apellido>
</nombre>
```

Elecciones, sólo se permite una sola de las opciones posibles presentadas. Las opciones, se separan por una barra vertical “|” y pueden ser dos o más:

```
<!ELEMENT artículos (articulo)+>
<!ELEMENT articulo (producto, identificador, (peso | cantidad))>
<!ELEMENT producto (#PCDATA)>
<!ELEMENT identificador (#PCDATA)>
<!ELEMENT peso (#PCDATA)>
<!ELEMENT cantidad (#PCDATA)>
.....
<artículos>
<articulo>
  <producto>sal</ producto >
  < identificador >8</ identificador >
  <peso>1.25</peso>
</articulo>
<articulo>
  <producto >vaso</producto>
  <identificador >123</identificador>
  <cantidad>10</cantidad>
</articulo>
<artículos>
```

Veamos un ejemplo en el que combinamos algunas de las opciones anteriores:

```
<!ELEMENT materiales (material)+>
<!ELEMENT material (elemento*| metal)>
<!ELEMENT elemento (#PCDATA)>
<!ELEMENT metal (#PCDATA)>
.....
< material >
  < elemento >cobre</elemento>
  < elemento >estaño</elemento>
</material >
```

```
<material>
    <metal>oro</metal>
</material>
```

Contenido mixto

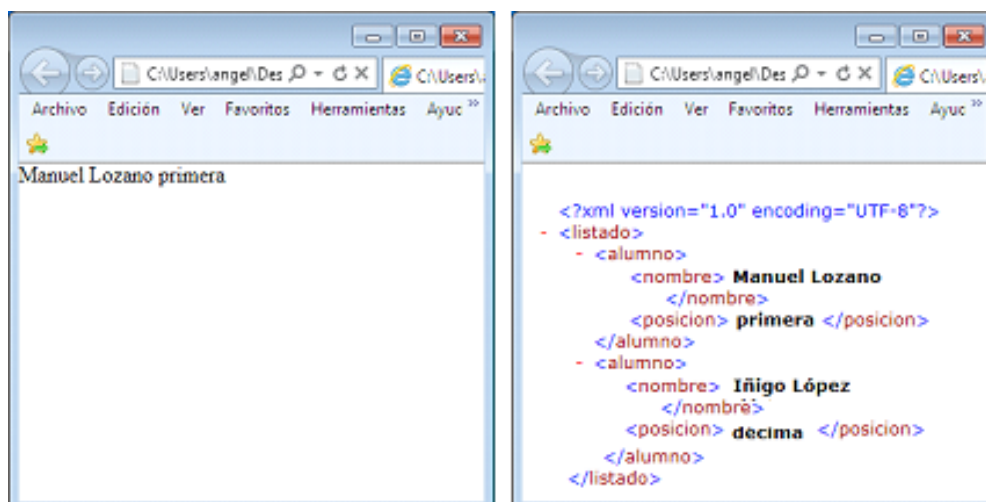
Ejemplo 1:

```
<!ELEMENT artículos (articulo)+>
<!ELEMENT articulo (#PCDATA | identificador)*>
<!ELEMENT identificador (#PCDATA)>
.....
<articulo>peras</articulo>
<articulo> <identificador>125-P</ identificador ></articulo>
<articulo> peras <identificador>125-P</ identificador ></articulo>
```

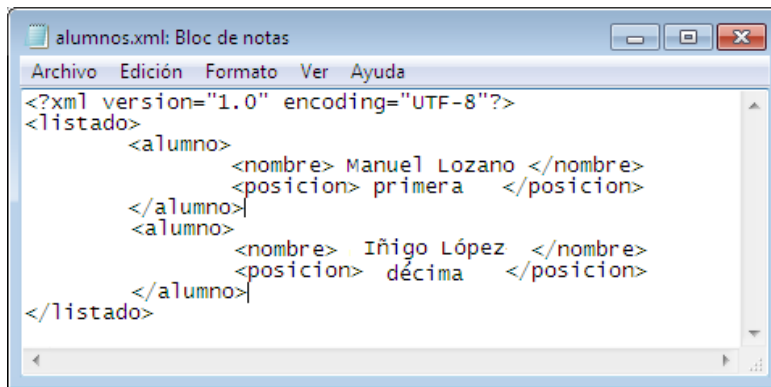
Ejemplo 2:

```
<!ELEMENT p (#PCDATA | b)*>
<!ELEMENT b (#PCDATA)>
.....
<p>Esta palabra es en <b>negrita</b> </p>
```

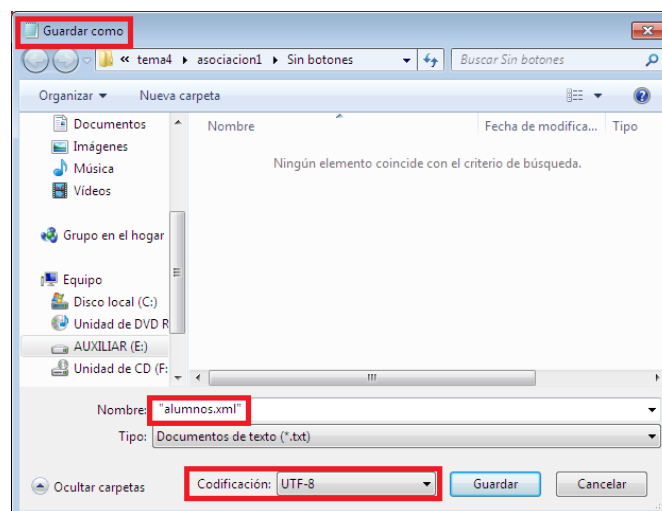
NOTA IMPORTANTE: para usar los contenidos XML con **acentos** y la letra "Ñ", debemos declarar que el tipo de codificación es UTF-8 de la siguiente forma `<?xml version="1.0" encoding="UTF-8"?>` pero teniendo además la precaución de guardar el archivo, no con la opción “guardar”, sino con **“guardar como”** y seleccionando que el formato de almacenamiento debe ser con **codificación UTF-8**. De otro modo aunque especifiquemos en la cabecera del código que la codificación es UTF-8, no se guardaría con esta codificación y daría error al abrirlo en el navegador. Observemos los dos resultados.



En ambos casos hemos escrito el mismo código que es éste:



Pero en el primer caso hemos dado a “guardar” en el block de notas y por defecto se ha guardado como ANSI, por eso el navegador no reconoce el documento como “bien formado” y no muestra correctamente la estructura jerárquica del mismo, sino que muestra el contenido HTML que es capaz de interpretar ya que los acentos y la ñ en este sistema de codificación no son reconocidos. Sin embargo, en el segundo caso hemos seleccionado “guardar como” y además hemos seleccionado que el archivo se debe almacenar como UTF-8, por eso el navegador sí que reconoce su estructura XML y la muestra admitiendo que está bien formado. Aunque paradójicamente los dos códigos sean idénticos, han sido almacenados con informaciones de codificación diferentes y el navegador los intenta interpretar de formas diferentes aunque se trate del mismo código.



Atributos

- Parejas Nombre/Valor
- Agregan información adicional a las etiquetas de inicio

<!ELEMENT comprador EMPTY>

<!ATTLIST comprador apellido CDATA nombre CDATA fecha CDATA
compra CDATA precio CDATA identificador CDATA>

.....
<comprador apellido="Martinez" nombre="Manuel"
fecha="21/03/2011" compra="Manzanas"
precio="3" identificador="T225" />

Declarando atributos en DTDs

La sintaxis a la hora de definir atributos es la siguiente, en la que el “tipo” corresponde a alguno de los vistos a continuación y el valor por defecto es el que queremos que tenga el atributo por omisión:

```
<!ATTLIST elemento
    atributo1 tipo valor_por_defecto
    atributo2 tipo valor_por_defecto
    .....
    atributo3 tipo valor_por_defecto >
```

Ejemplo en el que definimos un único atributo *lenguaje* (cuyo valor por defecto es “Español”) al elemento *mensaje*:

```
<!ELEMENT mensajes (mensaje)+>
<!ELEMENT mensaje (#PCDATA)>
<!ATTLIST mensaje lenguaje CDATA "Español">
.....
<mensajes>
  <mensaje lenguaje="Español"> ¡Hola! </mensaje>
  <mensaje> ¡Adios! </mensaje>
  <mensaje lenguaje="Ingles"> ¡Hello! </mensaje>
</mensajes>
```

Tipos de Atributos

- **CDATA** - Simple (Character Data) - Texto que no lleva marcas

```
<!ELEMENT circulo EMPTY>
<!ATTLIST circulo radio CDATA "0px">
.....
<circulo radio="40px"/>
```

- Tipo simbólico:

NMTOKEN - Una palabra que puede contener letras, números, punto [.], guión [-], subrayado [_] y dos puntos [:]. No puede contener espacios en blanco.

```
<!ELEMENT nombre_estudiante (#PCDATA)>
<!ATTLIST nombre_estudiante num_estudiante NMTOKEN #REQUIRED>
.....
<nombre_estudiante num_estudiante="12345">
  Albert Einstein
</nombre_estudiante >
```

NMTOKENS - Palabras que pueden usar los mismos elementos que en NMTOKEN pero además permiten espacios en blanco.

```
<!ATTLIST empleado fecha NMTOKENS #REQUIRED>
.....
<empleado fecha="27 02 1977">
  Javier Perez
</empleado>
```

ID

- Identificador único del atributo para cada elemento
- De tipo #REQUIRED o #IMPLIED (obligatorio u opcional)
- No debe aparecer más de una sola vez en el documento XML
- El primer carácter debe ser una letra o '_', o ':'
- Los demás caracteres no pueden ser espacios.

```
<!ELEMENT estudiante (#PCDATA)>
<!ATTLIST estudiante estudiante _id ID #REQUIRED>
```

```
.....
<estudiante estudiante _id="E50716">
    Angel Rodriguez
</estudiante>
```

IDREF - Referencia hacia el ID de otro elemento. Debe coincidir con el valor de algún atributo de tipo ID en algún elemento del documento.

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE grupo [
    <!ELEMENT grupo (estudiante)*>
    <!ELEMENT estudiante (#PCDATA)>
    <!ATTLIST estudiante estudiante _id ID #REQUIRED
        suplente IDREF #IMPLIED>
]>
```

```
.....
<grupo>
<estudiante estudiante _id ="E1123">Ana Garcia</estudiante>
<estudiante estudiante _id ="E1124">Isabel Lopez</estudiante>
<estudiante estudiante _id ="E1125" suplente="E1123">Maria Sanchez</estudiante >
</grupo>
```

IDREFS - Referencia a varios ID de otros elementos. Deben estar separados por espacios en blanco y deben coincidir con valores de atributo de tipo ID pertenecientes a elementos del documento. Por tanto, ahora será válido cualquiera de los valores especificados en el atributo de tipo IDREFS.

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE grupo [
    <!ELEMENT grupo (estudiante)*>
    <!ELEMENT estudiante (#PCDATA)>
    <!ATTLIST estudiante estudiante _id ID #REQUIRED
        suplentes IDREFS #IMPLIED>
]>
.....
<grupo>
<estudiante estudiante _id ="E1123">Ana Garcia</estudiante>
<estudiante estudiante _id ="E1124">Isabel Lopez</estudiante>
<estudiante estudiante _id ="E1125" suplentes="E1123 E1124">Maria
Sanchez</estudiante >
</grupo>
```

- **Tipo enumerado** - Listas de valores (se debe usar alguno de ellos, cualquier otro valor daría error)

```
<!ELEMENT dia (#PCDATA)>
<!--ATTLIST dia momento (mañana | tarde | noche) "tarde"-->
.....
<dia>Ejemplo de atributo con valores enumerados</dia>
<dia momento="noche">Programming XML in Java</dia>
```

En este ejemplo si se omite el atributo, tomaría por defecto el valor “tarde”. Como ocurre en el primer elemento “dia”.

Declaración predeterminada, valor por defecto.

Es el tercer y último componente requerido en la definición de un atributo. Tiene cuatro posibles formatos:

- **"VALOR"**

En este caso, hemos decidido usar un valor por defecto (default) que será el especificado en "VALOR" (entre comillas): `<!--ATTLIST elemento atributo CDATA "VALOR"-->`. De esta forma si no se usa un valor para el atributo, éste tomará el valor especificado en "VALOR".

```
<!ELEMENT img EMPTY>
<!--ATTLIST img
      alt CDATA "imagen"
      src CDATA "ruta" -->
```

```
.....

```

En este caso el atributo “src” toma el valor asignado “imagen.jpg” pero como no se ha usado el atributo “alt” éste toma el valor por defecto que es “imagen”.

- **#REQUIRED**

No hay un default (valor por defecto), pero el atributo del documento debe indicar algún valor necesariamente

```
<!ELEMENT img EMPTY>
<!--ATTLIST img
      alt CDATA #REQUIRED
      src CDATA #REQUIRED-->
```

```
.....

```

- **#IMPLIED**

No hay un default (valor por defecto), y el atributo del documento no necesariamente tiene que utilizarse

```
<!ELEMENT img EMPTY>
<!--ATTLIST img
      alt CDATA #REQUIRED
```

src CDATA #REQUIRED
width CDATA #IMPLIED
height CDATA #IMPLIED>

.....

- **#FIXED VALUE**

Se fija el valor del atributo al valor "VALUE".

<!ELEMENT direccion (#PCDATA)>
<!ATTLIST direccion ciudad CDATA #FIXED "Madrid">
.....
< direccion ciudad="Madrid">Calle General Ricardos 111.</direccion>

Entidades (Entities)

Las ENTIDADES resultan muy útiles cuando se va a repetir información o bloques grandes de texto que pueden aparecer varias veces dentro de un documento XML. Por ello los bloques o elementos a repetir suelen estar guardados en archivos separados o si se trata de una expresión corta puede declararse en el propio documento. El nombre abreviado, por ejemplo "bloque" que representa al bloque o expresión que se repite, aparecerá las veces que sean necesarias en el documento, pero irá generalmente precedido de un "&" y seguido de un punto y coma (;) en el documento XML (&bloque;).

Como hemos dicho el texto a representar a través de una variable, puede estar en el propio documento (en la DTD) o si es más extenso puede estar en un archivo externo. Por eso, la primera clasificación que podemos hacer de las entidades es:

- Entidad interna - Definida en el documento XML (en su DTD interna).
- Entidad externa - Definida en una fuente externa: archivo, URI (dirección web).

Pero por otro lado, puede ocurrir, que el elemento representado y que se repite, esté en el documento XML o en la propia definición de la DTD (cosa quizá menos usual). Entonces para diferenciarlas tenemos entidades generales y de parámetro:

- Entidad General
 - Se usa en el contenido de un documento.
 - Las referencias empiezan con '&' y terminan con ';'.
- Entidad de parámetro
 - Se usa en una DTD.
 - Las referencias empiezan con '%' y terminan con ';'.

Integrando todo lo anterior, nos encontramos con las siguientes situaciones:

Entidad General Interna

Formato: <!ENTITY nombre definicion> referencia &nombre;

Ejemplo:

```
<!ELEMENT fecha (#PCDATA)>
<!ENTITY hoy "30 de Abril de 2013">
<fecha>&hoy;</fecha>
```

General Externa

Formato: <!ENTITY nombre SYSTEM URI> o <!ENTITY nombre PUBLIC FPI URI> referencia &nombre;

(FPI: es el identificador público formal, recordemos que es una cadena de texto que sirve para identificar una especificación o documento)

Ejemplo:

```
<!ELEMENT fecha (#PCDATA)>
<!ENTITY hoy SYSTEM "fecha.xml">
.....
<fecha>&hoy;</fecha>
```

Referencia de Carácter (caso particular de entidad general)

Una referencia de carácter puede tener dos formatos:

- 	 donde 9 es uno o más dígitos decimales (de 0 a 9) que corresponde al código numérico del carácter en el conjunto ISO/IEC 10646.
- &#xh; donde h es uno o más dígitos hexadecimales (de 0 a F) que corresponde al código numérico del carácter en el conjunto ISO/IEC 10646.

- **&**; - El símbolo & con referencia de carácter & o &
- **'**; - El símbolo ' con referencia de carácter ' o '
- **>**; - El símbolo > con referencia de carácter > o >
- **<**; - El símbolo < con referencia de carácter < o <
- **"**; - El símbolo " con referencia de carácter " o "

```
<!ELEMENT correo (#PCDATA)>
<!ENTITY nuevo "&#64;">
.....
<correo>carlos&nuevo;mail.udlap.mx</correo>
```

De Parámetro

Formato:

- Interna
<!ENTITY %nombre definicion> referencia %nombre;.
- Externa
<!ENTITY %nombre SYSTEM URI> referencia %nombre;.
<!ENTITY %nombre PUBLIC FPI URI> referencia %nombre;.

Ejemplo:

```

<!ELEMENT vendedor (nombre, fecha)>
<!ELEMENT comprador (nombre, fecha)>
<!ENTITY % marca "(nombre,fecha)">
<!ELEMENT vendedor %marca;>
<!ELEMENT comprador %marca;>

```

1.1 ESPACIOS DE NOMBRES

En un documento XHTML el elemento raíz html presenta este aspecto:

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es">
```

El atributo xmlns es obligatorio, y su valor es el URL indicado. ¿Qué es esto? Es el *espacio de nombres*.

Como ya se introdujo en el tema anterior, para nuestra sorpresa, la mayoría de los espacios de nombres no existen, en el sentido de que el URL especificado, no apunta a ninguna parte.

El espacio de nombres no es más que un identificador que agrupa una serie de elementos pertenecientes a la misma aplicación XML, esto permite diferenciar dos elementos que compartan el mismo nombre, pero que tengan una semántica diferente en lenguajes distintos.

Veamos un ejemplo, supongamos que tenemos un documento que necesita contener información matemática. Como XHTML no tiene elementos con los que expresar elementos matemáticos, necesitamos de otro lenguaje de marcado XML como MathML. Como ambos son aplicaciones XML, su sintaxis es la misma, sólo se diferencian en los elementos y atributos que existen en cada uno de ellos. Gracias a los espacios de nombres, podemos identificar elementos y crear documentos mixtos.

Al espacio de nombres se le asigna un prefijo que después acompaña a cada elemento y que así los diferencia. Basta con definir en el elemento raíz del documento todos aquellos espacios que se necesiten. Vayamos un poco más lejos, y pensemos que necesitamos además que nuestro documento realice gráficos vectoriales. Ahora necesitaremos emplear marcado XHTML, MathML y SVG. Este sería el código:

```

<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1 plus MathML 2.0 plus SVG
1.1//EN" "http://www.w3.org/2002/04/xhtml-math-svg/xhtml-math-svg.dtd">
<html xmlns:xhtml="http://www.w3.org/1999/xhtml"
      xmlns:svg="http://www.w3.org/2000/svg"
      xmlns:math="http://www.w3.org/1998/Math/MathML"
      xml:lang="es">
...

```

Vemos que es necesario indicar una DTD, que ya existe y es pública, y que agrupa estos tres lenguajes de marcado, y a continuación indicamos los espacios de nombres, eligiendo los prefijos que luego precederán a cada elemento de cada uno de ellos. Ya podemos usar los elementos de cada lenguaje, identificándolos de forma única:

```

<?xml version="1.0" encoding="utf-8"?>
<?xml-stylesheet type="text/css" href="estilo.css"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1 plus MathML 2.0 plus SVG
1.1//EN" "http://www.w3.org/2002/04/xhtml-math-svg/xhtml-math-svg.dtd">

```

```

<html xmlns:xhtml="http://www.w3.org/1999/xhtml"
      xmlns:svg="http://www.w3.org/2000/svg"
      xmlns:math="http://www.w3.org/1998/Math/MathML"
      xml:lang="es">
<head>
  <meta http-equiv="Content-Type" content="application/xhtml+xml" />
  <meta name="language" content="es" />
  <meta http-equiv="Content-Language" content="es" />
  <title>Ejemplo de espacios de nombres</title>
  <link href="estilo.css" rel="stylesheet" type="text/css" />
</head>
<body>
  <h1>Fragmento de <b>SVG</b></h1>
  <p>Este ejemplo contiene código de <b>SVG</b></p>
  <svg:svg width="20cm" height="20cm" viewBox="0 0 180 180">
    <svg:desc>Un cuadrado y un círculo</svg:desc>
    <svg:defs>
      <svg:rect id="rectangulo" width="200" height="200" fill="red" x="10" y="10"
stroke="black" />
      <svg:circle id="circulo" r="100" cx="200" cy="200" fill="white"
stroke="black" />
    </svg:defs>
  </svg:svg>
  <h1>Fragmento de <b>MathML</b></h1>
  <p> Este ejemplo contiene código de <b>MathML</b></p>
  <math:mrow>
    <math:apply>
      <math:eq/>
      <math:ci>A</math:ci>
      <math:matrix>
        <math:matrixrow>
          <math:ci>x</math:ci> <math:ci>y</math:ci>
        </math:matrixrow>
        <math:matrixrow>
          <math:ci>z</math:ci> <math:ci>w</math:ci>
        </math:matrixrow>
      </math:matrix>
    </math:apply>
  </math:mrow>
</body>
</html>

```